Búsqueda de camino

Ambos algoritmos resuelven el mismo problema: dado un grafo dirigido con pesos positivos, hallar el camino de menor costo entre `inicio` y `destino`. En tu proyecto el grafo es una lista de adyacencia `{nodo: [(vecino, peso), ...]}`. Los sentidos únicos de los pasillos ya están codificados como aristas dirigidas.

Dijkstra (`PathFinderV.dijkstra`)

Idea

Mantiene un conjunto de nodos "cerrados", aquellos cuya distancia mínima desde el origen ya es definitiva. En cada paso cierra el nodo abierto más cercano al origen. Funciona porque, con pesos no negativos, ningún camino que pase por nodos más lejanos puede llegar al nodo más cercano con menor costo.

Estructuras del código

- `dist`: mejor costo conocido hasta cada nodo.
- `padre`: de qué nodo se llegó, para reconstruir el camino.
- `cola`: heap de `(distancia, nodo)`.
- `cerrados`: nodos ya definitivos.

Paso a paso

1. `dist[inicio] = 0` y se encola `(0, inicio)`.
2. Se saca el mínimo de la cola. Si ya estaba en `cerrados`, se descarta. Esto se llama *eliminación perezosa*: en vez de actualizar una entrada del heap, se inserta una nueva y la vieja se ignora al salir.
3. Se lo marca como cerrado. Si es el `destino`, se devuelve `(costo, camino, expandidos)` y se corta acá.
4. Si no, se **expande**: por cada vecino no cerrado se calcula `nueva = dist_actual + peso`. Si mejora `dist[vecino]`, se actualizan `dist` y `padre` y se encola. A esto se le llama *relajar* la arista.
5. Si la cola se vacía sin alcanzar el destino, devuelve `INFINITO` (no hay camino dirigido).

Complejidad

Cada nodo se expande a lo sumo una vez (el `cerrados` lo garantiza), así que se recorren a lo sumo E aristas.

Cada relajación exitosa hace un `heappush` de O(log n). Hay a lo sumo E pushes y E pops, entonces O(E log E) = O(E log V), porque $E \leq V^2$.

En la grilla el grado es ≤ 4, entonces E ≤ 4V y el total queda en **O(V log V)** de tiempo y **O(V)** de memoria.

El corte temprano no cambia el peor caso (destino lejano o inalcanzable), pero sí el caso típico.

Lectura de los resultados

Como no sabe dónde está el destino, explora un "disco" de radio igual a la distancia al destino. Los datos lo confirman: expande ~33% del área del almacén en las tres escalas (0,34 / 0,32 / 0,34), con pendiente 1,01 en el ajuste log-log. Es decir, Θ(V) nodos por búsqueda en la práctica.

A* (`PathFinderV.a_estrella`)

Idea

Es Dijkstra con una brújula. En vez de ordenar por `g(n)` (costo ya recorrido), ordena por

$$f(n) = g(n) + h(n)$$

donde `h(n)` estima el costo restante hasta el destino. Nuestra h es la distancia Manhattan: |Δx| + |Δy|. Así se priorizan los nodos que parecen acercarse al destino y se expanden muchos menos.

Condiciones que garantizan que el resultado sea óptimo

- **Admisible**: h nunca sobreestima el costo real. En una grilla con movimientos de 4 direcciones, Manhattan es el mínimo posible aun sin obstáculos, y los obstáculos y sentidos únicos solo pueden alargar el camino real.
- **Consistente**: h(n) ≤ peso(n, m) + h(m) para toda arista. Se cumple si cada arista pesa al menos la distancia Manhattan entre sus extremos. Esto vale tanto en el grafo completo (peso 1, distancia 1) como en el reducido (peso = largo del tramo recto).
- **Consecuencia práctica**: con h consistente, el primer momento en que se saca un nodo de la cola ya es su costo mínimo, así que nunca hay que reabrir nodos. Por eso nuestra implementación puede usar un conjunto `cerrados` simple, igual que Dijkstra.

Diferencias con Dijkstra

- La prioridad del heap es **`nuevo + manhattan(vecino, destino)`** en vez de **solo nuevo**.
- El costo real se guarda aparte en `costo`, porque la prioridad ya no es el costo.
- El chequeo `actual == destino` va antes de cerrar el nodo, y devuelve `costo[destino]`.

Complejidad

- **Peor caso**: O(V log V), igual que Dijkstra. Si h no aporta información (por ejemplo, un muro obliga a rodear todo el mapa), A* se comporta como Dijkstra.
- **Caso típico**: expande solo los nodos con f menor o igual al costo óptimo. Mientras mejor aproxime h al costo real, más cerca queda de expandir solo el camino.
- Cada nodo cuesta un poco más (calcular `manhattan` en cada push), pero eso es O(1).

Lectura de los resultados

- Nodos por búsqueda: 20 / 25 / 73 contra 64 / 96 / 424 de Dijkstra (básico / mediano / grande).
- Crece como ~V^0,7 (pendiente 0,70). La fracción explorada baja de 11% a 6% al agrandar el almacén.
- La razón de nodos va de 2,3× a 6,9×, pero la de tiempo solo de 0,6× a 5,5× (mediana 1,7×). En escenarios chicos el costo de calcular h puede anular el ahorro.
- La distancia total es casi igual porque ambos son óptimos por tramo. Las pequeñas diferencias vienen del desempate entre caminos de igual costo (la tupla `(f, nodo)` desempata por coordenadas, distinto a `(dist, nodo)`), lo que altera las colisiones entre pickers.

Cómo se usan en el sistema

- **Planificación (`PlanificadorViajesV`)**: se corre sobre el grafo reducido (`grafo_reducido`), que tiene menos nodos (esquinas, accesos y proyecciones). Las búsquedas pasan por `_buscar_con_cache`, que guarda cada par `(grafo, origen, destino)`.
- **Movimiento (`Navegacion.camino`)**: se corre sobre el grafo completo, con `GrafoFiltrado` para excluir las celdas ocupadas por otros pickers. Una búsqueda por cada destino candidato.
- Las métricas `nodos_expandidos` y `busquedas_camino` del experimento salen del planificador.

Escala	Área (celdas)	Nodos expandidos por Búsqueda (A* / Dijkstra)	Razón	Tiempo de Simulación (A* / Dijkstra)
Básico	188	20,3 / 64,1	3,2x	3,8 / 5,6 ms
Mediano	298	24,7 / 95,6	3,9x	18,2 / 28,4 ms

Grande	1241	72,6 /423,7	5,8x	226 / 687 ms

2. Estrategias de recorrido

Las tres reciben los m nodos de acceso del viaje y devuelven el orden de visita. Cada tramo se resuelve con A* o Dijkstra. El costo total es (cantidad de búsquedas) × S. Además, `evaluar_viaje` repite la estrategia para cada celda candidata de la zona de despacho.

Orden

Visita los nodos en el orden en que aparecen en la wave, sin deduplicar más allá de `dict.fromkeys`. Ese orden refleja prioridad y pedido, no geometría.

Búsquedas: m+1, es decir O(m·S).

Es la línea base. Su calidad depende de si el orden casualmente es espacial.

Cercanía (vecino más cercano)

Desde la posición actual busca a todos los pendientes y va al más cercano. Repite hasta terminar y vuelve al despacho.

Búsquedas: $m + (m-1) + \dots + 1 \approx m^2/2$, es decir $O(m^2 \cdot S)$.

Es una heurística voraz, sin garantía de óptimo. Con pasillos de sentido único la matriz de distancias es asimétrica, así que ni siquiera hay una cota de aproximación clásica.

Zonas

Clasifica cada rack por orientación (`ancho > alto` es horizontal). Ordena los verticales por (x, y) y los horizontales por (y, x), y recorre verticales y luego horizontales.

Costo: O(m log m) de ordenamiento más m+1 búsquedas, o sea O(m log m + m·S). Asintóticamente igual a Orden.

No usa la posición del picker ni el sentido de los pasillos. Siempre ordena de menor a mayor, aunque el pasillo vaya en sentido contrario.

Estrategia	Distancia media	Ticks	Búsquedas	Tiempo de simulación
Cercanía	**1080**	**320**	**414**	**176 ms**
Zonas	**1274**	**380**	**371**	**183 ms**
Orden	**1301**	**386**	**392**	**194 ms**

- **Cercanía recorre menos:** por escenario, Orden hace en promedio 31% más distancia (mediana 24%) y Zonas 25% más (mediana 18%).
    - Cercanía fue mejor o igual que Orden en 37 de 39 escenarios.
    - Cercanía ganó a Zonas en 33 de 39.
    - Zonas ganó a Orden en 24 de 39. Es una mejora real pero modesta.
- **El término O(m²) casi no se nota:** Cercanía hace solo ~6% más búsquedas que Orden, no ~m/2 veces más. Mi explicación es que `_buscar_con_cache` reutiliza pares (origen, destino) entre subconjuntos del backtracking, y que m es chico porque las waves tienen pocas unidades (15 por defecto). Esto lo infiero del código y de los conteos, no lo medí directamente.
- **Cercanía termina siendo la más barata en tiempo:** con menos ticks hay menos pasos de simulación, y eso compensa sobradamente las búsquedas extra.
- **Cuándo no gana:** con capacidades de wave de 1 pedido o 3 unidades, el viaje tiene 1–2 nodos y las tres estrategias coinciden (razón 1,00). Orden ganó por 4% en el layout “espina con orientaciones alternadas”. Su peor caso relativo fue 2,18× en `basico_variacion_pickers_baja_1`.
- **Dispersión de productos:** la correlación entre dispersión y penalización de Orden salió negativa (−0,41). Esperaba lo contrario, así que no lo usaría como conclusión sin más experimentos.

Conclusiones

A* es la mejor opción por defecto. Mantiene la optimalidad por tramo y reduce nodos 3–7×, con ventaja creciente según V. Dijkstra solo sería preferible si la heurística no fuera consistente.

Cercanía es la mejor estrategia: más complejidad teórica (O(m²)) pero menos distancia y menos ticks, y el costo extra queda absorbido por la caché.

Zonas ahorra poco frente a Orden. Probablemente mejoraría si respetara el sentido de cada pasillo (recorrido en serpentina) y partiera desde la posición del picker.